



Habib University
shaping futures

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 08 - Part 1

Operator Overloading, Rule of 3, Pointers to Objects





THIS WEEK'S AGENDA

- 01. Overloading Assignment and Comparison
- 02. Overloading Extraction and Insertion Operators
- 03. RAI
- 04. Rule of 3
- 05. Pointers to Class Objects





ASSIGNMENT AND COMPARISON





COMPARISON OPERATORS

- It is also possible to overload the ==, !=, and other comparison operators. For example:

```
bool operator==(const Distance &other) const {  
    return feet == other.feet && inches == other.inches;  
}  
  
bool operator!=(const Distance &other) const {  
    return !(*this == other);  
}
```





EXTRACTION AND INSERTION OPERATORS





EXTRACTION AND INSERTION OPERATORS

- Here ostream represents the output stream, and the function accepts an output stream as a parameter along with a user-defined class object to be sent to the output stream.
- It then returns the output stream by reference with the user-defined object printed.
- Let's look at an example:

```
21  std::ostream& operator <<(std::ostream& out, Distance &d) {  
22      out << "Feet: " << d.getFeet() << ", Inches: " << d.getInches();  
23      return out;  
24  }
```





EXTRACTION AND INSERTION OPERATORS

- We can make use of this overloaded extraction operator in the following way:
 - `Distance d1(5, 5.12);`
 - `std::cout << d1 << std::endl;`
- What this really means is:
 - `(std::cout << d1) << std::endl;`
- And the function is called with these arguments:
 - `operator <<(cout, d1);`

```
21  std::ostream& operator <<(std::ostream& out, Distance &d) {  
22      out << "Feet: " << d.getFeet() << ", Inches: " << d.getInches();  
23      return out;  
24  }
```





EXTRACTION AND INSERTION OPERATORS

```
26  std::istream& operator >>(std::istream &in, Distance &d) {  
27      int feet;  
28      double inches;  
29      std::cout << "Enter the feet and inches: ";  
30      in >> feet >> inches;  
31      d = Distance(feet, inches);  
32      return in;  
33  }
```

```
Distance d1;  
std::cin >> d1;
```





RAII





RESOURCE ACQUISITION IS INITIALIZATION

- RAI is a design principle that simplifies resource management by ensuring that:
 - Resources are always acquired at the point of initialization.
 - Resources are properly released when the object's lifetime ends, regardless of how the scope is exited (even if due to exceptions).
- Also known as Scope-Bound Resource Management (SBRM).
- The most common example of RAI is the management of dynamic memory.
- In the case of classes, RAI can be implemented with the usage of constructors and destructors to allocate and de-allocate resources provided to the class objects.





KEY FEATURES OF RAI

1. ***Automatic Resource Management:*** The responsibility of releasing resources is handled by destructors, so that there's no need to manually manage the resource lifecycle.
2. ***Exception Safety:*** Resources are correctly freed even if an exception is thrown, ensuring no memory or resource leaks occur.





RULE OF 3





THE RULE OF 3

- The rule of 3 is another design principle that states that if a class requires a user-defined destructor, a user-defined copy constructor, or a user-defined copy-assignment operator then it almost certainly requires all three.
- Recall that the default copy constructors and assignment operators create a shallow copy.
- Therefore, if a class has a user-defined copy constructor and assignment operator then that means we want to create a deep copy of our objects.





THE RULE OF 3

- Suppose that our class does not define a copy constructor. This means that copying an object will copy all of its data members to the target object.
- When the time comes to destroy the original object, the destructor will actually be invoked twice, and will have the same information for each object (original and copied).
- Therefore, in the absence of an appropriately defined copy constructor, the destructor is executed twice when it should actually only execute once.
- This duplicate execution can become a problem and lead to memory leaks or unwanted object deletions.





THE RULE OF 3 - DESTRUCTORS

- Recall that a **destructor** is a function that is
 - called when an object goes out of scope or,
 - is explicitly deleted.
- It is responsible for cleaning up resources that the object may have acquired during its lifetime, such as dynamically allocated memory.





THE RULE OF 3 - COPY CONSTRUCTORS

- A constructor that creates a new object as a copy of an existing object.
- It is called when an object is passed by value, returned from a function, or explicitly copied.
- If the class manages resources (like dynamic memory), a **default copy constructor**, will lead to **shallow copies**, causing resource management issues.
- What is a shallow copy? It is creation of an alias of a dynamically allocated object.





THE RULE OF 3 - COPY ASSIGNMENT OPERATOR

- This is an overloaded = operator.
- It is used to copy the contents of one existing object to another existing object.
- Like the copy constructor, if the class manages resources, the **default assignment operator**, will lead to shallow copies.





COPY CONSTRUCTOR VS COPY ASSIGNMENT OPERATOR

- **Copy Constructor:** A special constructor that initializes a new object as a copy of an existing object.
 - `ClassName(const ClassName &other);`
 - Called when a **new object** is created from an existing object.
- **Copy Assignment Operator:** A special assignment operator that assigns the contents of one existing object to another existing object (both objects are already created).
 - `ClassName& operator =(const ClassName& other);`
 - Called when an **existing object** is assigned to another existing object.



RULE OF 3 - EXAMPLE

```
1  #include <iostream>
2  #include <cstring>
3
4  class MyString {
5      private:
6          char* data;
7      public:
8          // Constructor
9          MyString(const char* str = "") {
10             data = new char[strlen(str) + 1];
11             strcpy(data, str);
12         }
13
14         // Destructor
15         ~MyString() {
16             delete[] data;
17         }
18     }
```

```
19         // Copy Constructor
20         MyString(const MyString &other) {
21             data = new char[strlen(other.data) + 1];
22             strcpy(data, other.data);
23         }
24
25         // Copy Assignment Operator
26         MyString& operator=(const MyString &other) {
27             if (this == &other) {
28                 return *this; // handle self-assignment
29             }
30             delete[] data; // free existing resource
31
32             data = new char[strlen(other.data) + 1];
33             strcpy(data, other.data);
34             return *this;
35         }
36
37         void print() const {
38             std::cout << data << std::endl;
39         }
40     };
```



RULE OF 3 - EXAMPLE

```
42  int main() {  
43      MyString str1("Hello, World!");  
44      MyString str2 = str1; // Calls copy constructor  
45      MyString str3;  
46      str3 = str1; // Calls copy assignment operator  
47  
48      str1.print();  
49      str2.print();  
50      str3.print();  
51  
52      return 0;  
53  }
```





RULE OF 3 - EXAMPLE

- Some of the problems that can occur when the rule of 3 is not followed are:
 - **Double Deletion:** deleting an object that has already been deleted due to the default copy constructor/copy assignment operator behavior.
 - **Shallow Copy:** creating shallow copies of objects, leading to a modification in all of them when only one of them is modified.
 - **Memory Leaks:** improper deletion and release of resources may cause memory leaks.





POINTERS TO CLASS OBJECTS





THE → NOTATION

- In C++, the → symbol is called the arrow operator and is primarily used to access members (attributes/data or methods) of an object or a struct through a pointer.
- When you have a pointer to an object or a struct, instead of dereferencing the pointer and then accessing the member variable or function using the dot (.) operator, you can instead use the → operator.
- Lets say we have the Node class with the member variable called value, as shown in the previous slide.





THE → NOTATION

- Then, if you have a pointer to a Node object, you can access its members as follows:
 - `Node *node1 = new Node;`
 - `node1->value = 15;`
 - `std::cout << "Node1 value: " << node1->value << std::endl;`
- Without using the arrow operator, this would be:
 - `(*node1).value = 15;`





SUMMARY





SUMMARY

- We looked at overloading the << and >> operators and also the assignment and comparison operators.
- We then discussed two design principles: RAI and Rule of 3
- The Rule of 3 states that: if a class needs to define one of the following:
 - Destructor, Copy Constructor, Copy Assignment Operator, then
- it needs to explicitly define all 3 in order to have exception-safe coding.
- Without Rule of 3, problems such as:
 - Double Deletion, Shallow Copies, and Memory Leaks
- may happen.





READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 08





ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science

